Title : 2. JSX and Component Creation Using Props.

App.jsx :

```
import Greeting from './Greeting'

const App = () => {
  return (
    <div>
      <Greeting name="Aaditya" />
      <Greeting name="Pranav" />
      <Greeting name="Parth" />
    </div>
  )
}

export default App
```

Greeting.jsx :

```
const Greeting = (props) => {
  return (
    <div>
        <h3>Hello, {props.name}...!!!</h3>
    </div>
  )
}

export default Greeting
```

Title : 3. State Management in React Using use State: Building a Counter App

App.jsx :

```
import { useState } from 'react'

const App = () => {
  const [count, setCount] = useState(0)
  return (

    <div className="flex flex-col justify-center items-center m-26 w-3/5 bg-amber-100 rounded-2xl">

      <h1 className='m-2 text-5xl font-bold'>React Counter</h1>
      <div className='p-5 border-2 rounded w-10 m-2 h-8 flex justify-center items-center mt-4'>{count}</div>

      <div className='flex gap-10 m-4'>
        <button className='border-2 rounded px-5'
        onClick={()=> setCount(count+1)}>+</button>
        <button className='border-2 rounded px-5'
        onClick={()=> setCount(count-1)}>-</button>
        <button className='border-2 rounded px-5'
        onClick={()=> setCount(0)}>Reset</button>
      </div>
    </div>
  )
}

export default App
```

Title : 4 Event Handling in React: Managing Button Clicks and Input Changes

App.jsx :

```
import { useState } from 'react'

const App = () => {
  const [name, setname] = useState('')
  const [message, setmessage] = useState('')
  const changeName= (e)=>{
    console.log(e.target.value);

    setname(e.target.value)
  }
  const buttonClick = ()=>{
    setmessage(`Hello, ${name}`)
  }
  return (
    <div className='m-10'>
      <input
      type="text"
      placeholder='Enter Your Name...'
      value={name}
      onChange={changeName}
      className='border-2 rounded m-4'/> <br />

      <button onClick={buttonClick}
      className='border-2 rounded m-4 px-2'>Greet Me</button>
      <br />
      <p className='m-4 text-emerald-500 text-xl font-
bold'>{message}</p>
    </div>
  )
}

export default App
```

Title : 5. Rendering Dynamic Lists in React Using Array Mapping and Key Props

App.jsx :

```
import React from 'react'

const App = () => {
  const fruits = ['Apple', 'Banana', 'Cherry'];// array
  return (
    <div className='m-4'>
        <h1>FRUITS LIST</h1>
      <ul>
        {fruits.map((fruit, index) => (
          <li key={index}> {fruit}</li>
        ))}
      </ul>
    </div>
  )

}

export default App
```

Title : 6 Building Controlled Forms in React: Managing State and Implementing Basic Validation

App.jsx :

```
import { useState } from 'react';

const App = () => {
  const [inputValue, setInputValue] = useState('');
  const [submit, setsubmit] = useState("");
  const [error, setError] = useState("");

  function handleInputChange(event) {
    const value = event.target.value;
    setInputValue(value);
    if (value.trim() !== "") {
      setError("");
    }
  }

  function handleSubmit(event) {
    event.preventDefault();
    if (inputValue.trim() === "") {
      setError("Name is required.");
      setsubmit("");
      return;
    }
    setsubmit(`Hello , ${inputValue} your form is submitted`);
    setError("");
  }

  return (
    <form onSubmit={handleSubmit} style={{ margin: "50px" }}>
      <label>
        Name:
        <input
          type="text"
          value={inputValue}
          onChange={handleInputChange}
        />
      </label>
      <br />
      <button type="submit" className="border rounded bg-fuchsia-300
px-4 my-4">
        Submit
      </button>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <p>{submit}</p>
    </form>
  );
}
export default App
```

Title : 7 Styling React Components: Implementing CSS Modules and Inline Styles

App.jsx :

```
import styles from './Button.module.css';

const App = () => {
  return (
    <button
      className={styles.button} //module css
      style={{ backgroundColor: "rgba(154, 43, 252, 0.655)" }} //inline
css
      onClick={() => { alert("Button Clicked") }}>
      Click Me
    </button>
  );
}
export default App
```

Button.module.css :

```
.button {
    color: white;
    padding: 10px 20px;
    margin: 10px;
    font-weight: 500
}
```

Title : 8 Conditional styling in React using both CSS Modules and inline styles. (Toggle Button with Conditional Styling).

App.jsx :

```
import "./toggle.css";
import { useState } from "react";
const App = () => {
  const [isOn, setIsOn] = useState(false);

  const buttonStyle = {
    backgroundColor: isOn ? '#4CAF50' : '#f44336',
    color: 'white',
    padding: '12px 20px',
    border: 'none',
    borderRadius: '5px',
    fontSize: '16px',
    cursor: 'pointer',
    transition: 'background-color 0.3s ease',
  };

  return (
    <div className="container">
      <span className="label">Toggle is {isOn ? 'ON' : 'OFF'}</span>
      <button style={buttonStyle} onClick={() => setIsOn(!isOn)}>
        {isOn ? 'Turn OFF' : 'Turn ON'}
      </button>
    </div>
  );

}
export default App
```

Title : 9 Developing a Personal Portfolio Website with React and Bootstrap

App.jsx :

```
import Navbar from './Navbar';
import Home from './Home';
import About from './About';
import Projects from './Projects';
import Footer from './Footer';
function App() {
  return (
    <div>
      <Navbar />
      <Home />
      <About />
      <Projects />
      <Footer />
    </div>
  );
}
export default App;
```

Navbar.jsx:

```
function Navbar() {
    return (
        <nav className="navbar navbar-expand-lg navbar-dark bg-dark">
            <div className="container">
                <a className="navbar-brand">My Portfolio</a>
                <div className="collapse navbar-collapse">
                    <ul className="navbar-nav ms-auto">
                        <li className="nav-item"><a className="nav-
link" href="#home">Home</a></li>
                        <li className="nav-item"><a className="nav-
link" href="#about">About</a></li>
                        <li className="nav-item"><a className="nav-
link" href="#projects">Projects</a></li>
                    </ul>
                </div>
            </div>
        </nav>
    );
}
export default Navbar;
```

Home.jsx

```
function Home() {
    return (
        <section id="home" className="bg-light text-center py-5">
            <div className="container">
                <h1>Hello, I’m Aaditya </h1>
                <p>Frontend Developer | React Enthusiast</p>
            </div>
        </section>
    );
}
export default Home;
```

About.jsx

```
import React from 'react';
function About() {
    return (
        <section id="about" className="py-5">
            <div className="container">
                <h2>About Me</h2>
                <p>I am a passionate web developer with skills in
React, Bootstrap, and modern web
                    technologies.</p>
            </div>
        </section>
    );
}
export default About;
```

Project.jsx

```
function Projects() {
    return (
        <section id="projects" className="bg-light py-5">
            <div className="container">
                <h2>Projects</h2>
                <ul>
                    <li>Portfolio Website</li>
                    <li>To-do App with React</li>
                    <li>Weather App using API</li>
                </ul>
            </div>
        </section>
    );
}
export default Projects;
```

Footer.jsx

```
function Footer() {
    return (
        <footer className="bg-dark text-white text-center py-3">
            <p>&copy; 2025 My Portfolio. All rights reserved.</p>
        </footer>
    );
}
export default Footer;
```

Title : 10 Setting Up a Node.js and Express.js Development Environment: A Practical Guide

Step 1: Setup the Backend (Node.js + Express.js)

1.Go to your folder on the top write cmd

2. In cmd write this commands one by one

- mkdir fullstack-app
- cd fullstack-app
- mkdir backend
- cd backend
- npm init –y
- code .
- in VS code your folder is open and create file server.js copy the code in server.js

2. Install Express

```
npm install express cors
```

3. Create server.js

add this code in server.js

```
const express = require('express');
const cors = require('cors');
const app = express();
const PORT = 5000;
app.use(cors());
app.use(express.json());
// Dummy API
app.get('/api/message', (req, res) => {
 res.json({ message: "Hello from the backend!" });
});
app.listen(PORT, () => {
 console.log(`Server running on http://localhost:${PORT}`);
});
```

4. Run the Backend Server

Run this command on CMD :
node server.js
Visit: http://localhost:5000/api/message
You should see:
{ "message": "Hello from the backend!" }

Title : 11. Implementing Basic Routing in Express.js: Creating Multiple GET Endpoints"

index.js

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
    res.send('Welcome to the Home Page!');
});

app.get('/about', (req, res) => {
    res.send('This is the About Page.');
});

app.get('/contact', (req, res) => {
    res.send('Contact us at contact@gmail.com');
});

app.get('/services', (req, res) => {
    res.send('Here are our services: Web Development, App
Development, SEO.');
});

app.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});
```

Title : 12 **Building a RESTful API with Express.js: A Hands-On Guide to HTTP Methods on (Customer details).**

index.js

```
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());

let customers = [
    { id: 1, name: 'Aaditya', email: 'aadityapawar@gmail.com' },
    { id: 2, name: 'Yash', email: 'yashpatil@gmail.com' }
];

app.get('/customers', (req, res) => {
    res.json(customers);
});
app.get('/customers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const customer = customers.find(c => c.id === id);
    if (!customer) {
        return res.status(404).send('Customer not found');
    }
    res.json(customer);
});

app.post('/customers', (req, res) => {
    const { name, email } = req.body;
    const newCustomer = {
        id: customers.length + 1,
        name,
        email
    };
    customers.push(newCustomer);
    res.status(201).json(newCustomer);
});

app.put('/customers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const customer = customers.find(c => c.id === id);
    if (!customer) {
        return res.status(404).send('Customer not found');
    }

    customer.name = req.body.name || customer.name;
```

```
    customer.email = req.body.email || customer.email;

    res.json(customer);
});

app.delete('/customers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = customers.findIndex(c => c.id === id);
    if (index === -1) {
        return res.status(404).send('Customer not found');
    }
    const deletedCustomer = customers.splice(index, 1);
    res.json(deletedCustomer);
});

app.listen(port, () => {
    console.log(`Customer API is running at
http://localhost:${port}`);
});
```

Title : 13. Building RESTful APIs: Managing JSON POST Requests in Express.js (Employee Post Method).

index.js

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());

let employees = [
 { id: 1, name: 'Aaditya', role: 'Developer' },
 { id: 2, name: 'Yash', role: 'Editor' }
];

app.get('/employees', (req, res) => {
 res.json(employees);
});

app.post('/employees', (req, res) => {
 const newEmployee = req.body;

 if (!newEmployee.name || !newEmployee.role) {
 return res.status(400).json({ error: 'Name and role are required'
});
 }

 newEmployee.id = employees.length + 1;
 employees.push(newEmployee);
 res.status(201).json(newEmployee);
});

app.listen(port, () => {
 console.log(`Server running at http://localhost:${port}`);
});
```

Title : Implementing CRUD Operations (Students)Using RESTful API and HTTP Methods.

```
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
// Fake in-memory database
let students = [];
let nextId = 1;
// CREATE - Add new student
app.post('/students', (req, res) => {
 const student = { id: nextId++, ...req.body };
 students.push(student);
 res.status(201).json({ message: 'Student created', student });
});
// READ ALL - Get all students
app.get('/students', (req, res) => {
 res.json(students);
});
// READ ONE - Get student by ID
app.get('/students/:id', (req, res) => {
 const student = students.find(s => s.id == req.params.id);
 if (!student) {
 return res.status(404).json({ message: 'Student not found' });
 }
 res.json(student);
});
// UPDATE - Update student by ID
app.put('/students/:id', (req, res) => {
 const index = students.findIndex(s => s.id == req.params.id);
 if (index === -1) {
 return res.status(404).json({ message: 'Student not found' });
 }
 students[index] = { id: students[index].id, ...req.body };
 res.json({ message: 'Student updated', student: students[index] });
});
// DELETE - Delete student by ID
app.delete('/students/:id', (req, res) => {
 const index = students.findIndex(s => s.id == req.params.id);
 if (index === -1) {
 return res.status(404).json({ message: 'Student not found' });
 }
 const removedStudent = students.splice(index, 1);
```

```
 res.json({ message: 'Student deleted', student: removedStudent[0] });
});
app.listen(port, () => {
console.log(`Server running at http://localhost:${port}`);
});
```

Title : Implementing CRUD Operations (Students)Using RESTful API and HTTP Methods.

🔧 1. Install and Set Up MongoDB

a. Install MongoDB Server

- Go to https://www.mongodb.com/try/download/community
- Choose:
  - **Version**: Latest Stable
  - **Platform**: Windows (or your OS)
  - **Package**: MSI
- Install it **with "Complete" setup** and **enable MongoDB as a Service** during installation.

b. Start MongoDB

- Open **Services** (Windows + R → services.msc)
- Ensure **MongoDB Server (mongod)** is running.
  If not, start it manually.

🧭 2. Install MongoDB Compass (GUI)

- Download from https://www.mongodb.com/try/download/compass
- Install and open Compass.
- In the **connection string**, enter:
- mongodb://localhost:27017
- Click **Connect**.
- You'll see your local databases listed.

🧩 3. Create a Database and Collection

In **MongoDB Compass**:

1. Click **"Create Database"**
   - Database Name → teachersDB
   - Collection Name → teachers
2. Click **Create Database**.
3. Insert sample data manually or via code later.

Example sample document:

```
{
  "name": "Aaditya Pawar",
  "subject": "Computer Science",
  "experience": 3,
  "email": "aaditya@example.com"
}
```

💻 4. Set Up Your Node.js API Project

In your terminal:

```
mkdir teachers-api
cd teachers-api
npm init -y
npm install express mongoose cors dotenv
```

🧠 5. Create Project Structure

```
teachers-api/
│
├── server.js
├── .env
└── models/
    └── teacherModel.js
```

⚙️ 6. Configure MongoDB Connection

.env

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/teachersDB
```

---

server.js

```
import express from "express";
import mongoose from "mongoose";
import dotenv from "dotenv";
```

```
import cors from "cors";
import Teacher from "./models/teacherModel.js";

dotenv.config();

const app = express();
app.use(express.json());
app.use(cors());

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log("✅ MongoDB connected"))
  .catch(err => console.log(err));

app.get("/", (req, res) => res.send("Teachers API Running"));

// Create teacher
app.post("/teachers", async (req, res) => {
  try {
    const teacher = new Teacher(req.body);
    await teacher.save();
    res.status(201).json(teacher);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});

// Get all teachers
app.get("/teachers", async (req, res) => {
  const teachers = await Teacher.find();
  res.json(teachers);
});
```

```
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => console.log(`🚀 Server running on port ${PORT}`));
```

---

models/teacherModel.js

```
import mongoose from "mongoose";

const teacherSchema = new mongoose.Schema({
  name: { type: String, required: true },
  subject: { type: String, required: true },
  experience: { type: Number, default: 0 },
  email: { type: String, unique: true, required: true }
});

export default mongoose.model("Teacher", teacherSchema);
```

💉 7. Test Your API

Run:

```
node server.js
```

You should see:

✅ MongoDB connected

🚀 Server running on port 5000

Now test endpoints using:

- **Postman** (or Thunder Client in VS Code)
- **GET:** http://localhost:5000/teachers
- **POST:** http://localhost:5000/teachers
  Example body:
- {
- "name": "John Smith",
- "subject": "Mathematics",
- "experience": 5,

- "email": "john@school.com"
- }

📊 8. View Data in MongoDB Compass

- Go to teachersDB > teachers collection
- You'll see your inserted teacher documents appear instantly.

✅ Outcome

You've now built:

- A **MongoDB-powered Teachers API**
- Connected it using **Mongoose**
- Viewed and managed data visually in **MongoDB Compass**

Title : Mini Project on Inventory Management System.

📘 Step 2: Create Your Project Folder

In your terminal or VS Code:

```
mkdir inventory-system
cd inventory-system
```

📘 Step 3: Initialize Node Project

```
npm init -y
```

This creates a package.json file.

📘 Step 4: Install Dependencies

You only need **Express** and **Mongoose**:

```
npm install express mongoose
```

📘 Step 5: Create the Main File

Create a file named:

server.js

Paste this full working code inside (the improved version I gave):

```
const express = require('express');
const mongoose = require('mongoose');
const app = express();

app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/inventoryDB", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("✅ Connected to MongoDB"))
.catch(err => console.error("❌ MongoDB connection error:", err));
```

```
const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  category: String,
  quantity: { type: Number, required: true, min: 0 },
  price: { type: Number, required: true },
  supplier: String,
  updatedAt: { type: Date, default: Date.now }
});

const Product = mongoose.model('Product', productSchema);

// Get all products
app.get('/products', async (req, res) => {
  const products = await Product.find();
  res.json(products);
});

// Get single product
app.get('/products/:id', async (req, res) => {
  const product = await Product.findById(req.params.id);
  if (!product) return res.status(404).send('Product not found');
  res.json(product);
});

// Add new product
app.post('/products', async (req, res) => {
  try {
    const product = new Product(req.body);
    await product.save();
    res.status(201).json(product);
  } catch (err) {
```

```
    res.status(400).json({ error: err.message });
  }
});

// Update product
app.put('/products/:id', async (req, res) => {
  const product = await Product.findByIdAndUpdate(req.params.id, req.body, { new: true });
  if (!product) return res.status(404).send('Product not found');
  res.json(product);
});

// Delete product
app.delete('/products/:id', async (req, res) => {
  const result = await Product.findByIdAndDelete(req.params.id);
  if (!result) return res.status(404).send('Product not found');
  res.send('✅ Product deleted successfully');
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`🚀 Inventory API running at http://localhost:${PORT}`);
});
```

🗄️ Step 6: Start MongoDB

If MongoDB is installed as a service (Windows):

- Press **Win + R** → **services.msc**
- Find **MongoDB Server** → Right-click → **Start**

OR
If you installed MongoDB manually:

```
mongod
```

📘 Step 7: Run Your Server

Now start the Node.js app:

node server.js

✅ If everything's correct, you'll see:

✅ Connected to MongoDB

🚀 Inventory API running at http://localhost:3000

📘 Step 8: Test in Postman or Thunder Client

1️⃣ Add a Product (POST)

URL: http://localhost:3000/products

Method: POST

Body (JSON):

```
{
  "name": "Laptop",
  "category": "Electronics",
  "quantity": 10,
  "price": 65000,
  "supplier": "Dell"
}
```

→ You'll get a response like:

```
{
  "_id": "671a7ff4532a9b61a12c22d1",
  "name": "Laptop",
  "category": "Electronics",
  "quantity": 10,
  "price": 65000,
  "supplier": "Dell",
  "__v": 0
}
```

2️⃣ Get all products (GET)

http://localhost:3000/products

3️⃣ Update a product (PUT)

http://localhost:3000/products/<id>

Body:

```
{
  "quantity": 15
}
```

4️⃣ Delete a product (DELETE)

http://localhost:3000/products/<id>